

Getting Started with LangChain

Lucas Soares

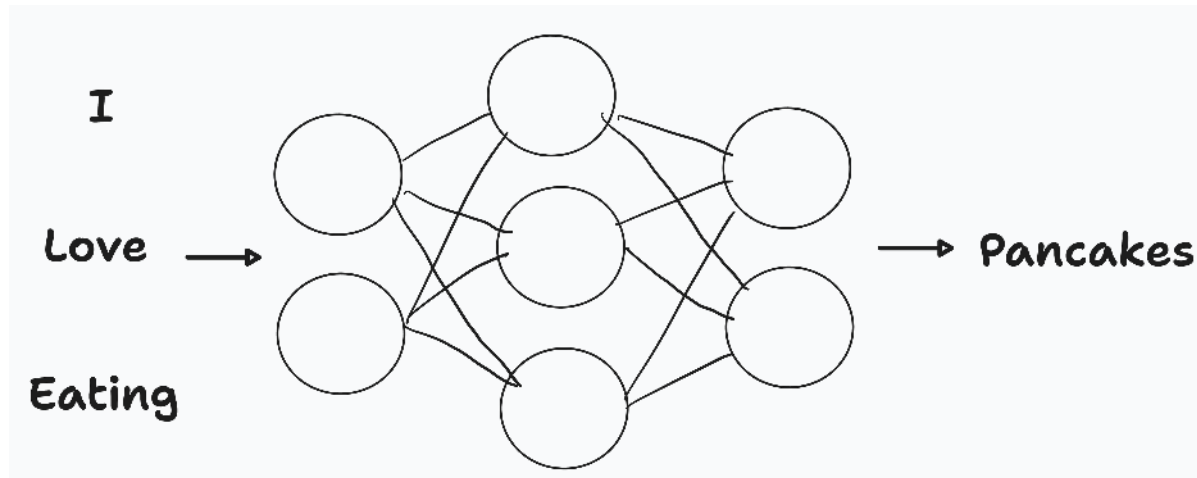
August 2026

Quick 'Interactivity Notes'

- The presentation will be organized into the following structure:
- **Presentation Block**
- **Notebook Demo**
- **Quick Q&A + Summary**
- **Optional Exercise** During Q&A (for those that don't have questions and want to try something out)
- Repeat

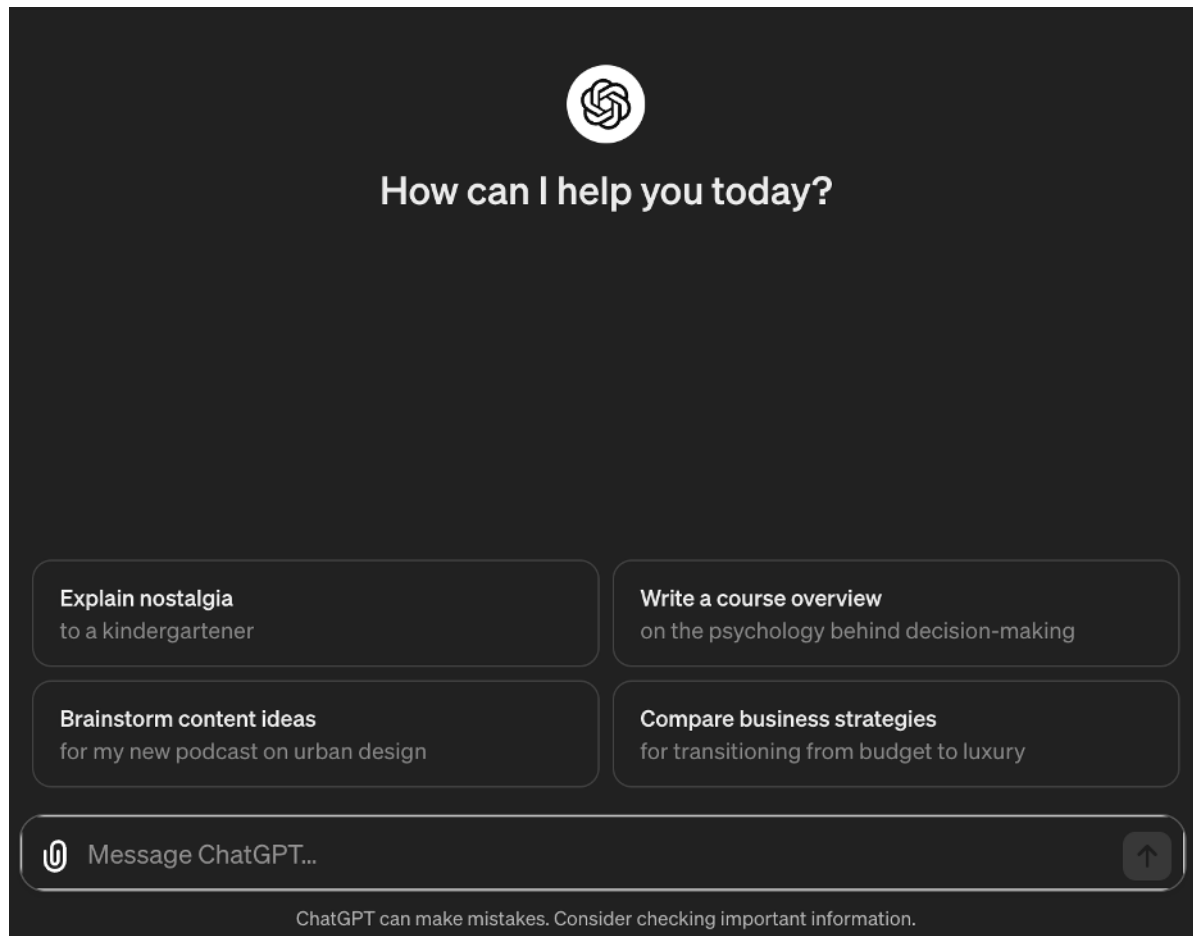
Large Language Models

Large Language Models Predict the Next Word



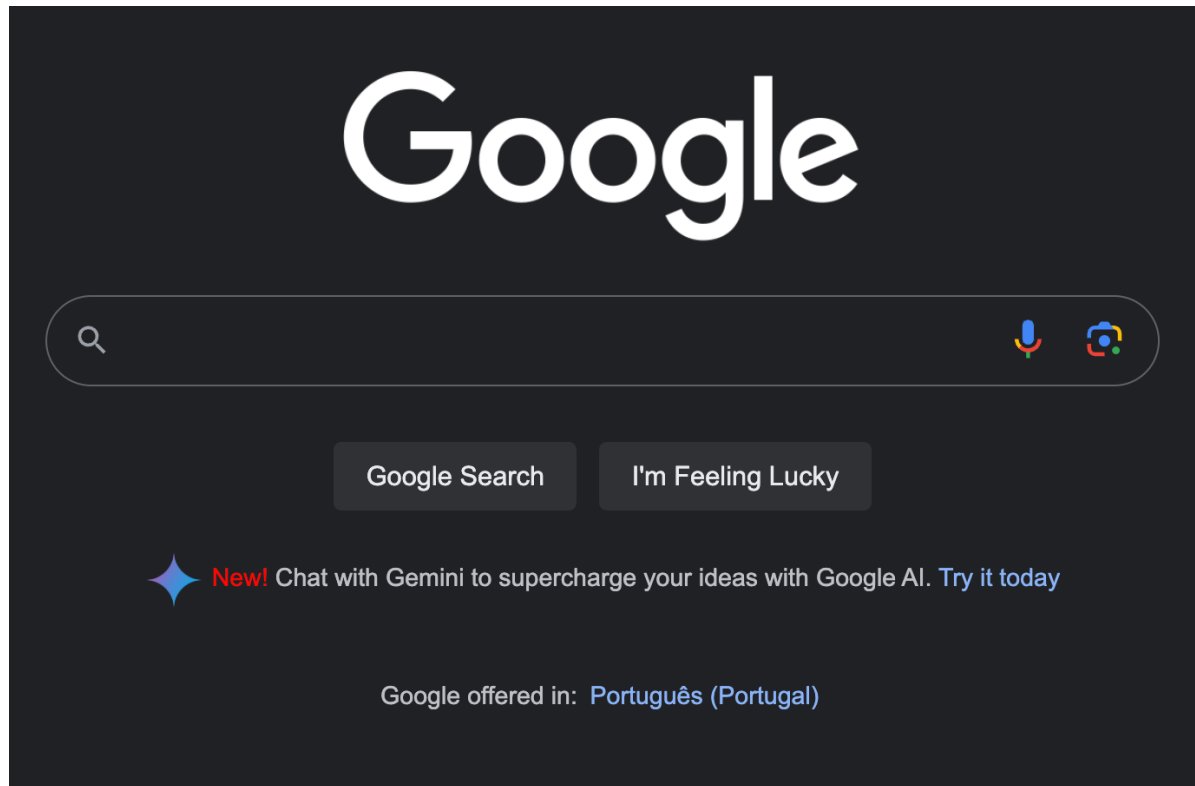
Applications of Large Language Models

- Conversational ChatBots



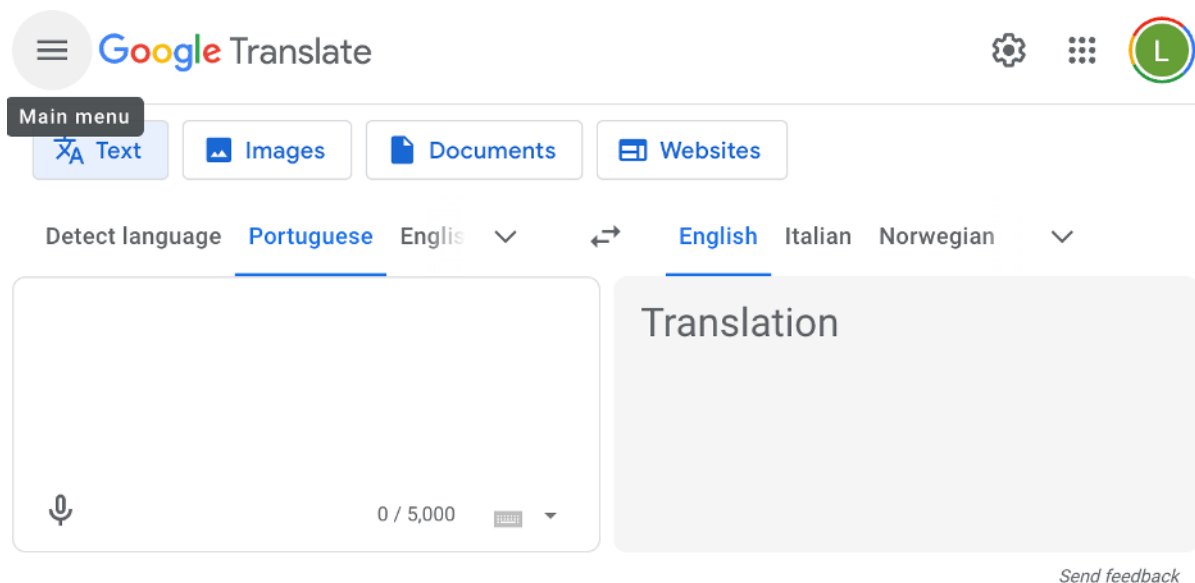
Applications of Large Language Models

- Search Engines



Applications of Large Language Models

- Translation



- And so much more from Q&A over PDFs to personalized tutoring.

What is Langchain?

- **LangChain framework for developing LLM-based applications**
- **Main features:**
 - components
 - prebuilt agents (`create_agent()`)

Whiteboard Explain - Building Blocks of Working with LLMs

LangChain 1.0 — Building Blocks

- **Agents** — `create_agent()`
- **Models** — `init_chat_model()`
- **Messages** — system, human, ai, tool
- **Tools** — `@tool` decorator
- **Short-term Memory + Streaming**

Models

- Provider-agnostic factory for any LLM

```
from langchain.chat_models import init_chat_model  
  
model = init_chat_model("gpt-5.6-terra")  
  
output = model.invoke("I am teaching a live-training about LLMs!")  
  
print(output.content)
```

- Same call shape for "openai:gpt-5.6-terra", "anthropic:claude-sonnet-5",
"google_genai:gemini-3.6-flash"

Messages

- Conversations are lists of typed messages:
- SystemMessage — sets behavior/persona
- HumanMessage — user input
- AIMessage — model response
- ToolMessage — tool execution result

```
from langchain.messages import HumanMessage, SystemMessage

model.invoke([
    SystemMessage(content="You tell jokes."),
    HumanMessage(content="Tell me a joke"),
])
```

Agents — create_agent()

- The central API in LangChain 1.0 — **replaces the old executor-based agent runtime**
- Returns a LangGraph CompiledGraph you call with {"messages": [...]}

```
from langchain.agents import create_agent

agent = create_agent(
    model,          # "provider:model-name" or a ChatModel
    tools=None,     # list of @tool-decorated functions
    system_prompt=None,
    checkpoint=CheckpointSaver, # for memory persistence
    response_format=None, # for structured output
)
```

Agents — Minimal Example

```
from langchain.agents import create_agent

# Minimal agent — 3 lines
agent = create_agent(model="openai:gpt-5.6-terra", tools=[])
result = agent.invoke({"messages": "What is LangChain?"})
result["messages"][-1].pretty_print()
```

- No manual chain piping — the agent decides what to do with the input

Why create_agent() First?

- No prompt | llm | output_parser piping to hand-wire
- Tools, memory, and streaming are built in from the start
- Works identically with zero tools or many tools
- The same API scales from a one-line chatbot to a full agent

Notebook Demo - Intro to LangChain

Q&A & Summary

- LLMs can predict the next word in a sequence. ("I Like eating...? ;P ")
- **LangChain framework:** eases the creation of LLM-based applications, built around **6 core building blocks**:

Agents

`create_agent()`

Models

`init_chat_model()`

Messages

Tools

Short-term Memory

Streaming

- `create_agent()` is the primary entry point — it replaces manually piping `prompt | llm | output_parser`.

Optional Exercise (during Q&A): Create a simple agent with `create_agent()` that summarizes content.

Break 5 minutes

Poll Question

Which statement best captures the primary advantage of using a framework like LangChain for LLM-based applications?

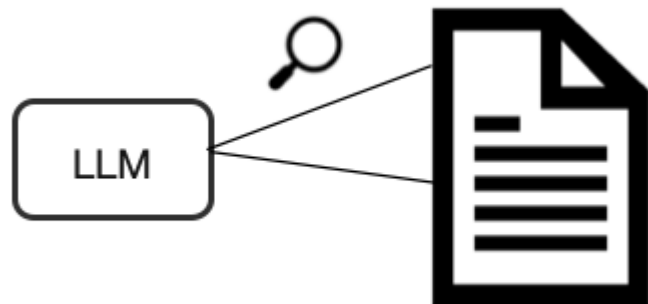
- A. It automatically trains your model without any configuration
- B. It offers unified abstractions (agents, models, tools) that streamline development
- C. It focuses solely on deployment, not on application logic
- D. It restricts user inputs to enhance security

Poll Question

Which best describes what `create_agent()` returns in LangChain 1.0?

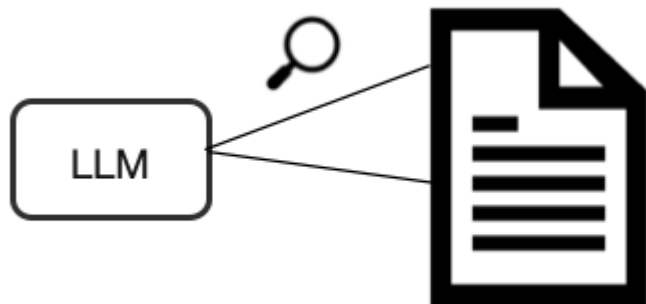
- A. A REST API endpoint for the model
- B. A compiled LangGraph graph you invoke with `{"messages": [...]}`
- C. A fixed script that cannot be modified once generated
- D. A raw string containing the model's response

LangChain for Chat Over Documents



- RAG = **R**etrieval **A**ugmented **G**eneration
- It's about connecting LLMs to documents like PDFs, Text files, HTML, etc.
- How do we get around the context length limitations of LLMs?
- Quick Answer is **Embeddings**!

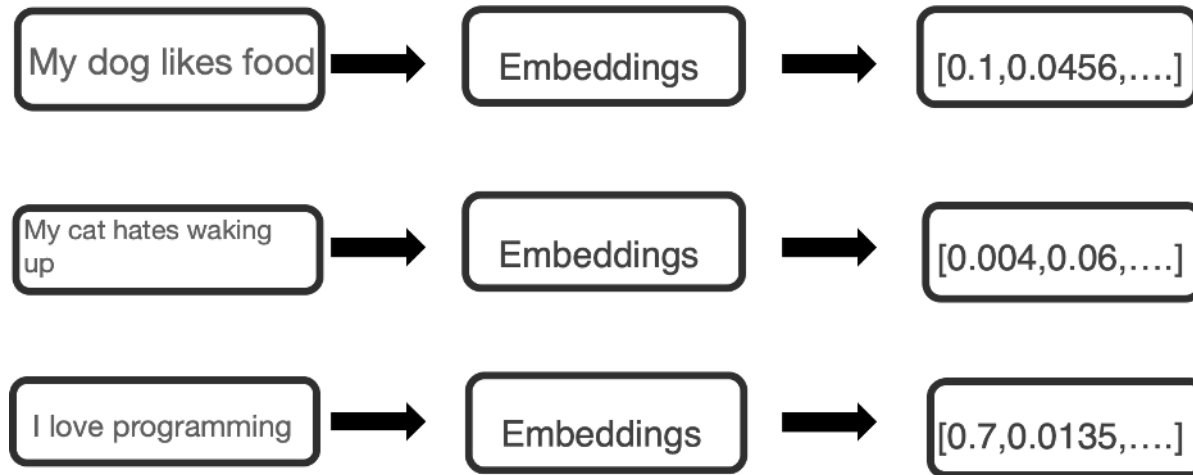
LangChain for Chat Over Documents



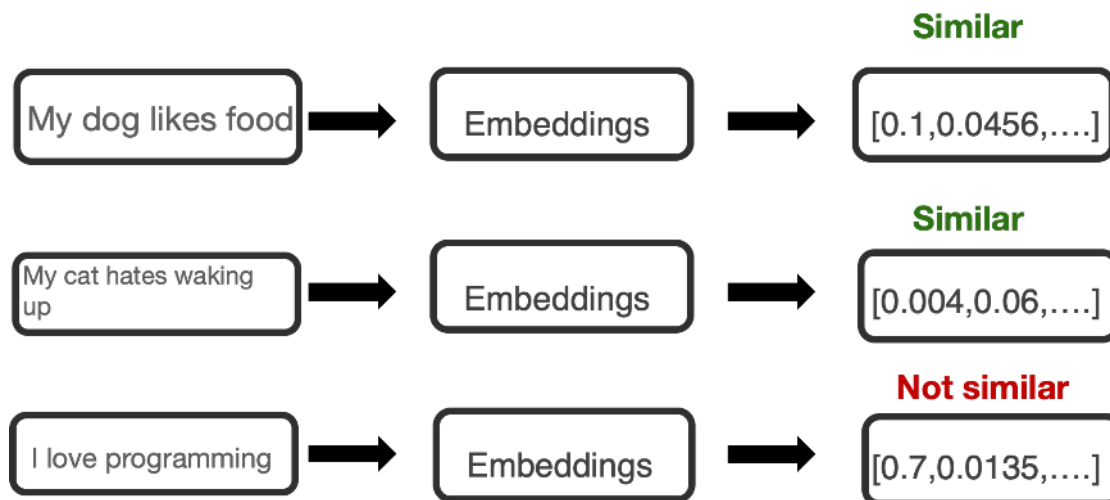
- Embeddings are vectorized representations of text



LangChain for Chat Over Documents

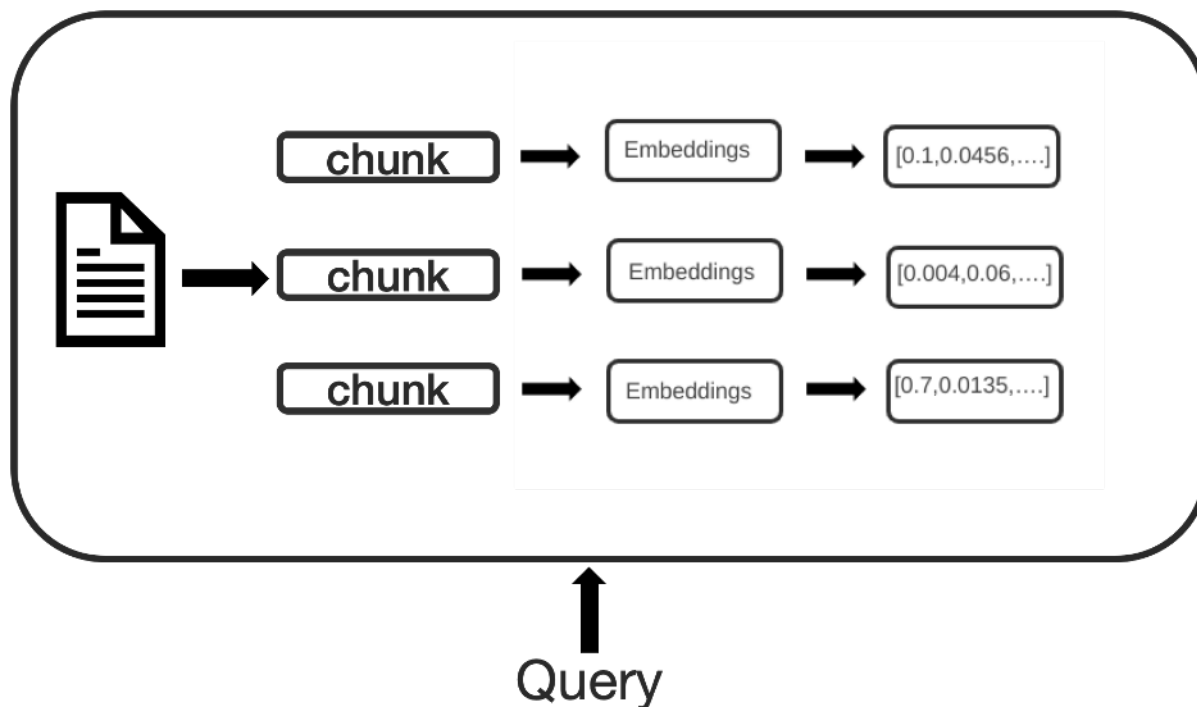


LangChain for Chat Over Documents



- Embeddings capture content and meaning

LangChain for Chat Over Documents



- Embeddings capture content and meaning
- Vector DBs
- How to build RAG systems with LangChain?

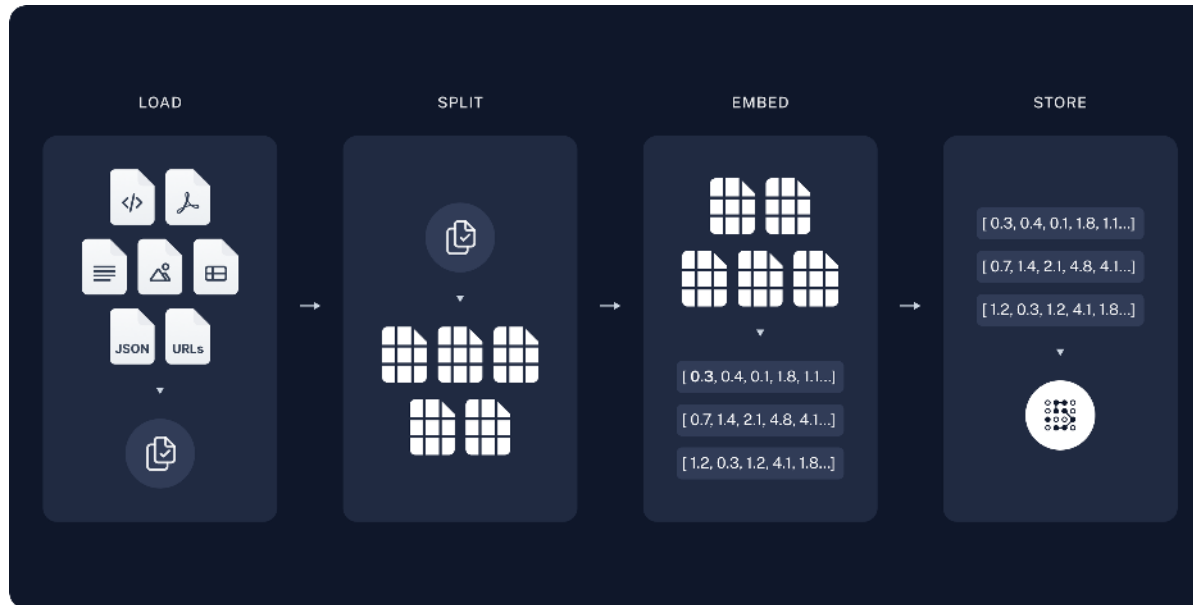
Whiteboard Explain - RAG Systems

Poll Question

What is the primary purpose of embeddings when building LLM-powered document chat systems?

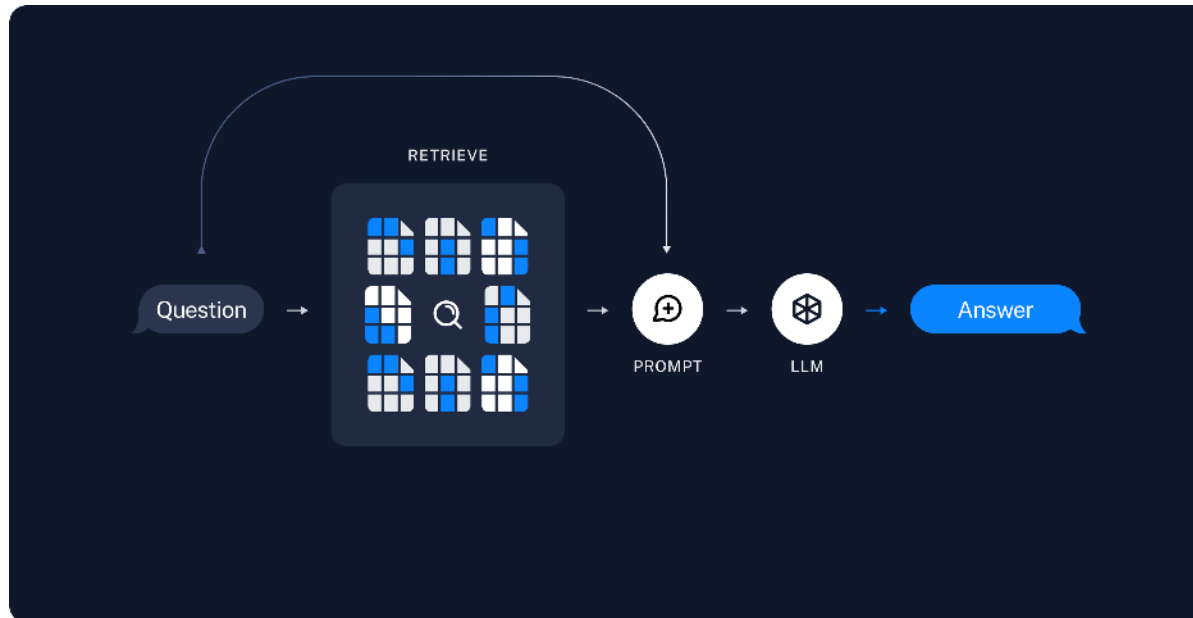
- A. Managing API requests and rate limits
- B. Turning text into numerical vectors for similarity comparison
- C. Automatically generating user interface elements
- D. Serving as a secure authentication method

LangChain for Chat Over Documents



- Load
- Split
- Embed
- Store

LangChain for Chat Over Documents



- Retrieval Pipeline
- Input Question
- Retrieve Relevant Documents
- LLM uses the prompt question + retrieved data to produce a final answer

LangChain for Chat Over Documents

Notebook Demo - Q&A with LangChain

Q&A & Summary

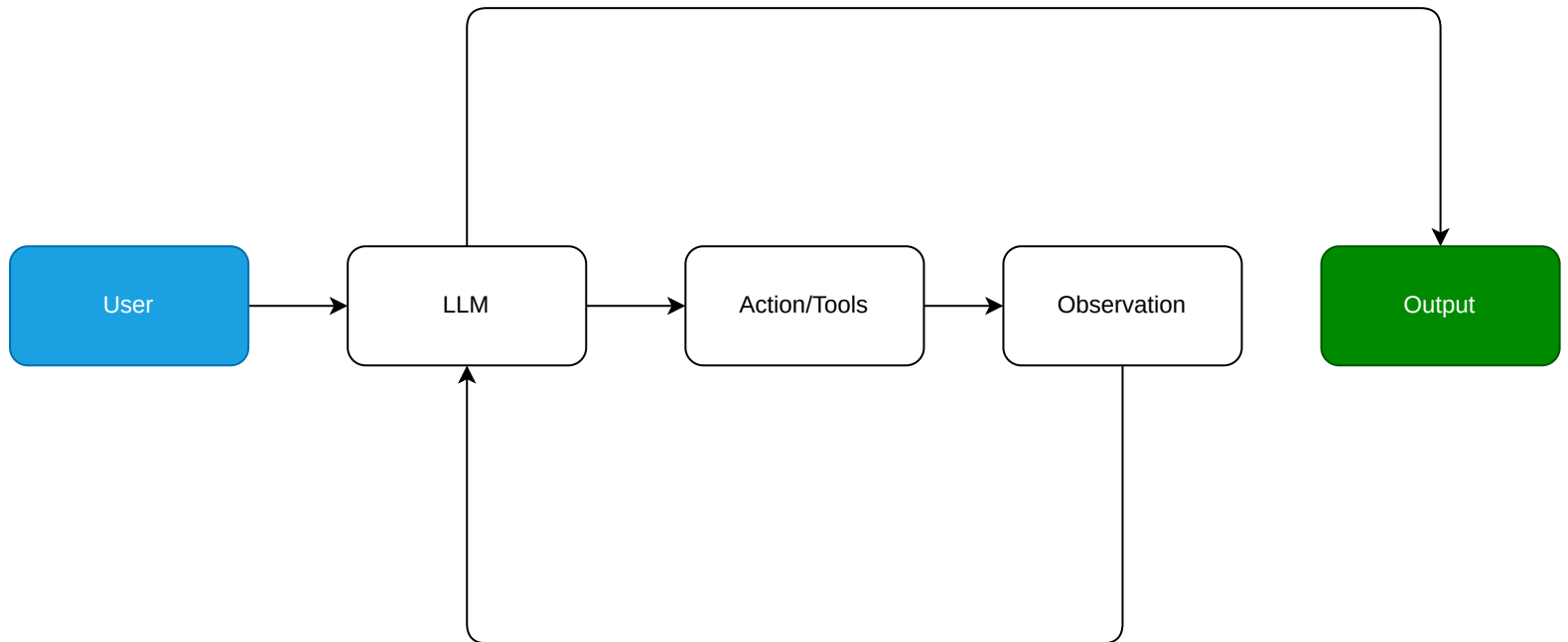
- **RAG** = **R**etrieval **A**ugmented **G**eneration — connects LLMs to documents like PDFs, text, HTML.
- **Embeddings** are vectorized representations of text that capture content and meaning.
- **Indexing pipeline**: Load → Split → Store (Chroma + OpenAIEmbeddings).
- **RAG Agents** (agent decides when to search, via a retrieval @tool) vs **RAG Chains** (always search, single LLM call).

Optional Exercise (during Q&A): Create a simple RAG agent with `create_agent()` that answers questions about pdfs or csvs.

Break 5 minutes

Building Agents with LangChain

The Agent Loop



Components of an Agent

- `create_agent()` takes a **model**, a list of **tools**, and a **system prompt**

```
model = init_chat_model("openai:gpt-5.6-terra", reasoning_effort="none")
agent = create_agent(
    model=model,
    tools=[read_file, write_file, search_tool],
    system_prompt="You are a helpful assistant...",
)
```

- It returns a **LangGraph agent** — no separate executor runtime needed
- Reasoning models (like gpt-5.6-terra) need `reasoning_effort="none"` to call tools

Tools — @tool Decorator

- Type hints and a docstring are mandatory — they define the schema the model sees

```
from langchain_core.tools import tool

@tool
def calculate(expression: str) -> str:
    """Evaluate a mathematical expression.

    Args:
        expression: A Python math expression like '2 + 2'
    """
    return str(eval(expression))
```

Memory & Streaming

- **Short-term memory:** pass a checkpointer (e.g. `InMemorySaver()`) + a `thread_id` — different threads, different memories
- **Streaming** modes: "values" (full state per step), "messages" (token by token), "custom" (progress updates from tools)

Poll Question

Which best describes the “Agent Loop” illustrated in the presentation?

- A. A direct data-loading pipeline for vector databases
- B. A continuous decision-making cycle where an LLM decides actions and checks results
- C. A script for parallel execution of tasks in a single pass
- D. A framework for chunking documents into smaller sections

Notebook Demo - Building an Agent with `create_agent()`

Q&A & Summary

- **create_agent()** builds an agent from a model, tools, and a system prompt — it replaces the old executor-based agent runtime.
- **Tools** are plain functions decorated with `@tool`; type hints and docstrings define the schema the model sees.
- **Short-term memory** comes from passing a checkpointer and a `thread_id`.
- **Streaming** has three modes: "values", "messages", and "custom".

Optional Exercise (during Q&A): Create a simple agent with `create_agent()` and a tool that builds a schedule given a table of tasks and deadlines.

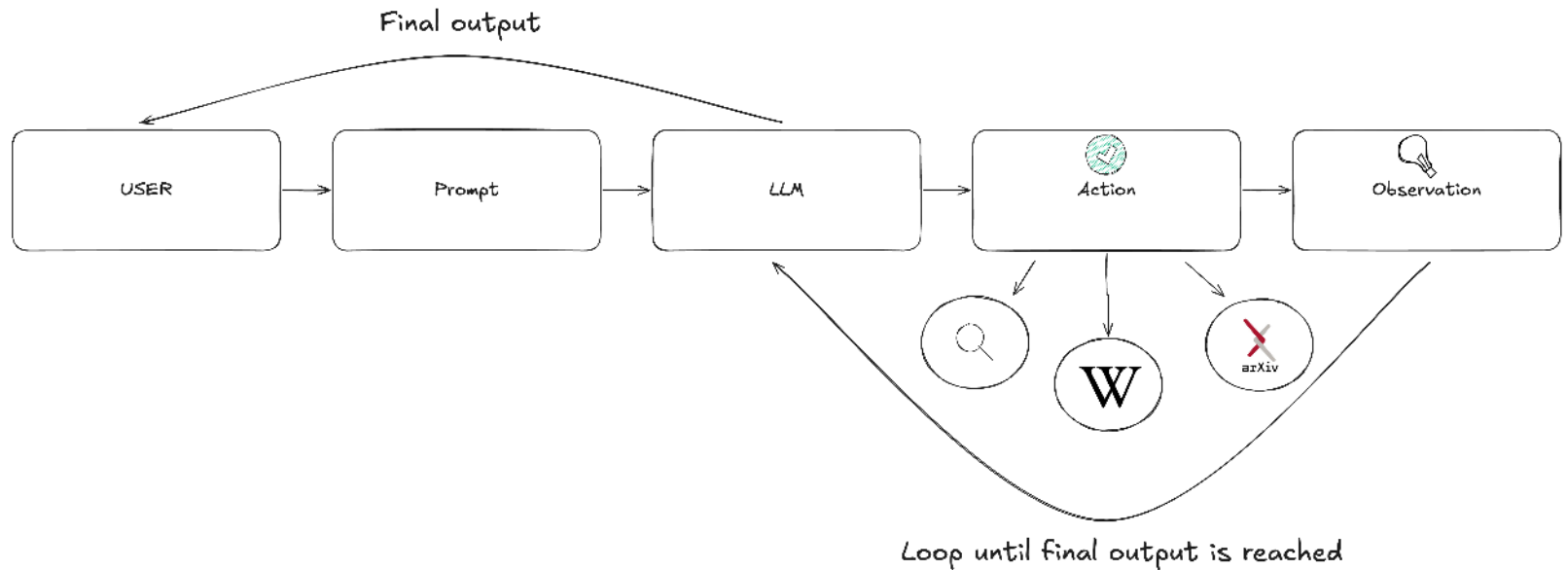
table format = task | date

Understanding Agentic Systems

- **Agentic Systems:** Systems that utilize large language models (LLMs) to manage the control flow of applications.
- **Key Functions:** Routing decisions, tool selection, and evaluating output sufficiency.
- **Agent Loop:** Continuous decision-making process that enables agents to solve complex tasks.

[Understanding agentic systems in LangGraph.](#)

The Agent Loop



[Explanation of the agent loop in cognitive architectures.](#)

Practical Use Case: Customer Support Agent

- **Scenario:** An LLM-powered customer support agent.
- **User Input:** Customer asks about order status.
- **LLM Decision:** Determines if it can provide the status directly or if it needs to fetch data from the database.
- **Action Taken:** If data fetch is needed, the agent queries the database and updates the user with the order status.

[Practical use case of LLM agents in customer support.](#)

Advantages of LLM Agents

- **Flexibility:** Agents can adapt to various tasks by determining the best action to take.
- **Specialization:** LLM agents can be specialized with a set of tools to perform niche tasks.
- **Multi-Agent Collaboration:** Specialized LLM agents can collaborate to perform complex tasks.

[LangGraph: Multi-Agent Workflows](#)

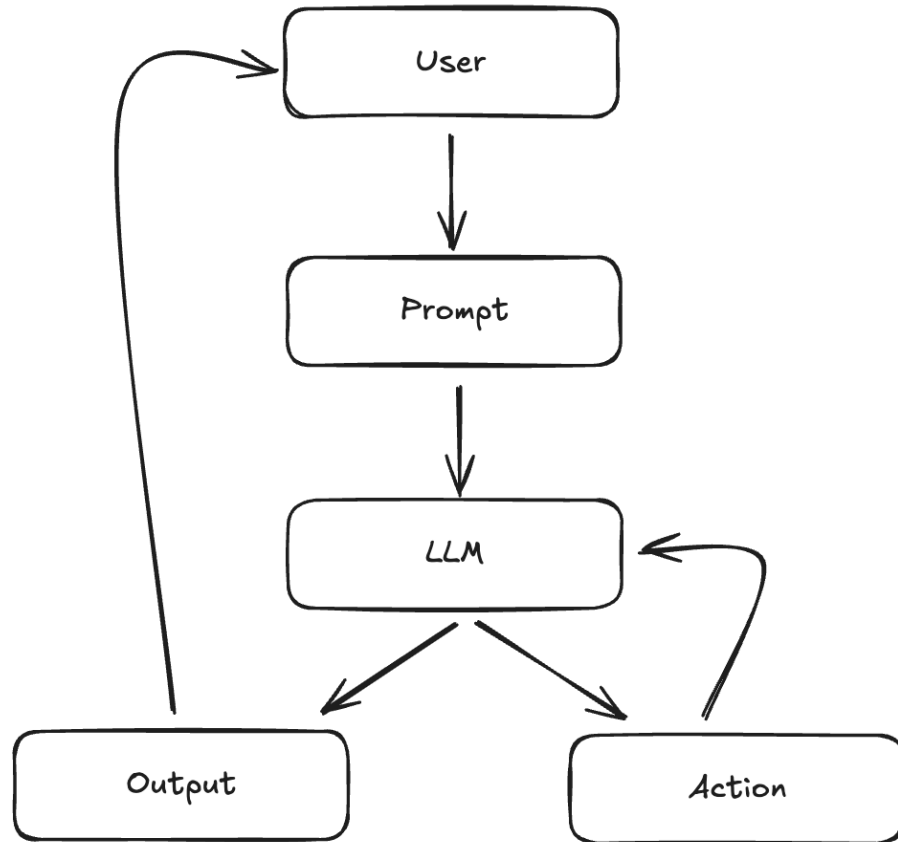
Key Components of Agentic Systems

- 1. **Tool Calling:** Utilizing external tools to perform tasks.
- 1. **Action Taking:** Executing actions based on LLM outputs.
- 1. **Memory:** Keeping track of interactions for context-aware responses.
- 1. **Planning:** Structuring steps to ensure optimal decision-making.

[Key components of agentic systems.](#)

Agents as Graphs

- Workflows built with agents are usually structured as graphs!!



Why LangGraph?

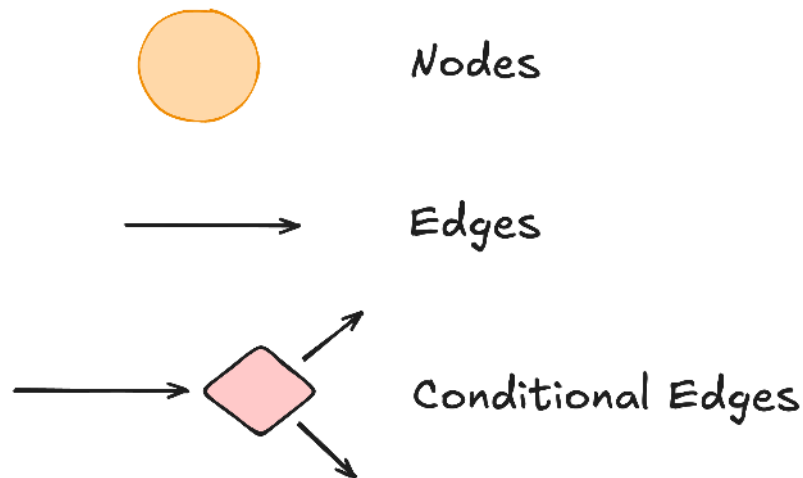
LangGraph is designed for building agentic applications with some core principles:

- **Controllability:** Offers low-level control which increases reliability in agentic systems.
- **Human-in-the-Loop:** Built-in persistence layer enhances human-agent interaction patterns.
- **Streaming First:** Supports streaming of events and tokens, providing real-time feedback to users.

[Overview of LangGraph's purpose and principles.](#)

The Basic Components of LangGraph

LangGraph models agent workflows as graphs:



- **Nodes:** Python functions that implement the logic of agents, taking the current State as input and returning an updated State.
- **Edges/Conditional Edges:** Functions that implement fixed/conditional transitions to determine which Node to execute next based on the current State.

[Explanation of LangGraph's components.](#)

States in LangGraph

These graphs in LangGraph are driven by:

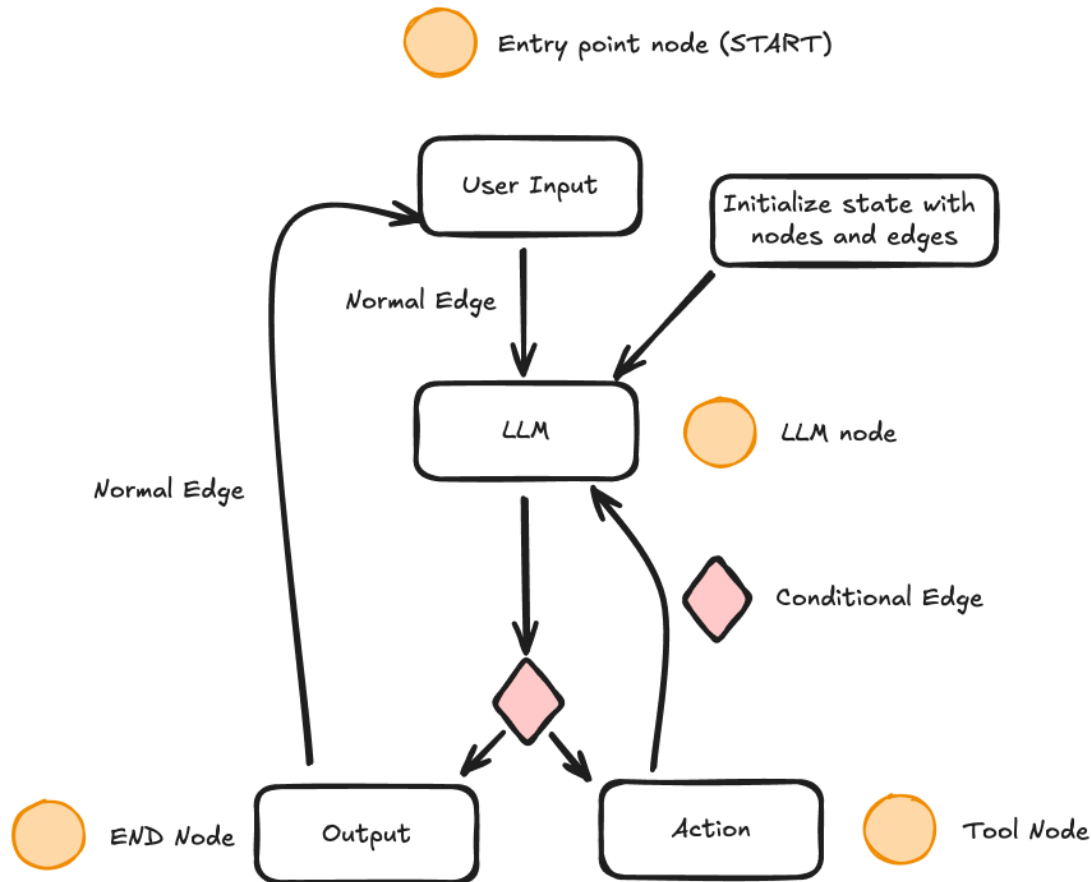
- **States:** Shared data structures that evolve over time as Nodes execute and pass messages along Edges.
- **Message Passing:** Nodes send messages to activate other Nodes, facilitating the execution of workflows in discrete iterations or "super-steps".

[Overview of how graphs and states interact in LangGraph.](#)

Notebook Demo: Introduction to LangGraph

Agent Loop in LangGraph

- Outline of a basic agent loop in langgraph:



Notebook Demo: A Basic Research Agent in LangGraph

Q&A & Summary

- **LangGraph** is a framework for building agentic applications.
- **Nodes** are the core functional units in LangGraph.
- **Edges** define the routing logic in LangGraph.
- **States** are shared data structures that evolve over time as Nodes execute and pass messages along Edges.
- **Agent Loop** is the continuous decision-making process that enables agents to solve complex tasks.

Break 5 minutes

Deploying Agents — langgraph dev

- Local development server with hot-reloading — starts at `http://localhost:2024`
- Run it from the repo's demo/ app: `uv run langgraph dev`
- Connect the **Agent Chat UI** at agentchat.vercel.app:
 - Deployment URL: `http://localhost:2024`
 - Graph ID: `chat_over_docs`
- Optional: enable `LANGSMITH_TRACING=true` to trace every run in LangSmith

Notebook Demo - Deployment with langgraph dev

Final Q&A & Summary

- **langgraph dev** starts a local server for any `create_agent()` agent — no separate deployment framework needed.
- The **Agent Chat UI** (agentchat.vercel.app) connects to it for interactive testing.
- The demo agent (`demo/agent.py`) is a RAG-over-docs agent — the same `create_agent()` + retrieval `@tool` pattern from the RAG section.
- **LangSmith** traces tool calls, token usage, and full conversation history when tracing is enabled.

References

- [LangChain Documentation](#)
- [LangChain Agents](#)
- [Gen Agents](#)
- [WebGPT](#)
- [OpenAI](#)
- [OpenAI Function Calling](#)
- [AutoGPT](#)
- [GPT-Engineer](#)
- [BabyAGI](#)
- [Karpathy on Agents](#)
- [ReACT Paper](#)

